

Rapport Individuel — Itération 2

IDE Java UML — Module Génération de Code

Nom : ESMAHI Salah eddine
Login : sei5471
Groupe : Groupe UML (Groupe 1)
Rôle : Conversion UML → Code Java
Date : Mai 2026

Contexte et objectif personnel

Cette itération fait directement suite à l'itération 1, qui avait permis de poser le modèle de données (`JavaClass`, `JavaAttribut`, etc.) et d'implémenter le générateur de code `JavaCodeGenerator`. Les trois points reportés en fin d'itération 1 constituaient l'agenda naturel de cette itération : gestion des relations, export vers fichier `.java`, et intégration avec l'interface graphique `JavaFX`.

Activités réalisées

Gestion des relations dans le générateur

J'ai étendu `JavaCodeGenerator` pour prendre en compte les relations UML stockées dans `JavaRelation`. Concrètement, la méthode `appendDeclaration` génère maintenant `extends` et `implements` selon le type de relation (`INHERITANCE` ou `REALIZATION`). Les relations de composition et d'agrégation sont quant à elles reflétées comme attributs dans le corps de la classe.

• Avancement : 100%

Résolution du conflit Git — `JavaClass.java`

Mon binôme avait ajouté la méthode `addRelation()` sur sa branche sans que je l'aie sur la mienne, ce qui a produit un conflit lors du merge. J'ai résolu manuellement le conflit en conservant la version avec `addRelation()`, puis commité et poussé :

```
git add JavaClass.java JavaMethode.java
git commit -m "fix: resolution conflit merge - ajout addRelation()"
git push origin ma-branche
```

• Avancement : 100%

Export vers fichier `.java`

J'ai implémenté `JavaFileExporter`, une classe utilitaire qui prend la chaîne produite par `JavaCodeGenerator` et l'écrit dans un fichier `.java` en utilisant `java.nio.file.Files`. Le nom du fichier est automatiquement déduit du nom de la classe.

- **Avancement : 100%**

Intégration avec l'interface `JavaFX`

J'ai coordonné avec mon binôme pour connecter le bouton « Générer le code » de l'interface `JavaFX` à mon générateur. Quand l'utilisateur clique, la classe sélectionnée dans le diagramme est passée à `JavaCodeGenerator`, et le résultat s'affiche dans une fenêtre `TextArea` tout en étant exporté sur le disque.

- **Avancement : 80%** (l'intégration fonctionne mais la synchronisation en temps réel entre le diagramme et le code n'est pas encore implémentée)

Bilan et difficultés rencontrées

Le conflit Git a été la principale surprise de cette itération. Travailler à plusieurs sur le même modèle sans se concerter sur les branches crée facilement des divergences. On a décidé de mieux se synchroniser en amont à partir de maintenant.

Sur le plan technique, la gestion des relations `implements` multiples a demandé un traitement particulier : il peut y avoir plusieurs interfaces implémentées, contrairement à `extends` qui ne peut avoir qu'une seule cible. J'ai adapté la méthode `appendDeclaration` pour accumuler les cibles `REALIZATION` dans une liste.

Reporté à l'itération 3

- Synchronisation en temps réel diagramme ↔ code Java
- Gestion des imports automatiques (`import` des types utilisés)
- Tests unitaires formels sur `JavaCodeGenerator` avec JUnit